

Specification for a Robust, WYSIWYG Video Persistence and Loading Experience

Part I: Architecting a High-Fidelity User Experience

This section establishes the user-centric vision for the new loading experience, grounding design decisions in Apple's Human Interface Guidelines (HIG) and the specific needs of "The Athlete" persona. The goal is to transform the loading screen from a passive waiting period into an informative, confidence-building part of the user journey.

1.1 Deconstructing the Current Experience: The Ambiguity of Indeterminacy

The current loading interface, which utilizes a gray spinner, presents an indeterminate progress indicator. According to Apple's Human Interface Guidelines, this approach is suboptimal for tasks where progress is quantifiable, such as file processing.¹ An indeterminate indicator offers no information regarding the task's duration, which can lead to user anxiety, frustration, and a perception that the application has stalled. For any operation that extends beyond a few seconds, users require more substantial feedback to remain engaged and confident in the system's operation.³ The current UI, as depicted in the provided image, compounds this issue by offering minimal context ("Preparing video import...") and a static percentage (10%), which can feel deceptive if the process does not advance smoothly.

This ambiguity has a significant negative impact on the target "Athlete" persona, who views the application as a professional tool for cataloging training footage. For this user, precision and control are paramount. An ambiguous loader is analogous to a faulty gauge on a high-performance machine; it undermines trust and suggests the system is unpredictable or unreliable.¹ When a user initiates the critical action of saving their training footage, the

system's presentation of an indeterminate spinner creates a void in their mental model. Questions such as, "Is it working?", "Is it stuck?", or "Will this take ten seconds or ten minutes?" introduce cognitive load and anxiety.⁴ This uncertainty increases the perceived wait time, as the user's attention is focused on the lack of progress rather than being productively distracted or informed.³ This experience is in direct opposition to the goal of a frictionless, "mechanical watch" precise user experience. The failure is not merely in the choice of a UI component but in its inability to manage user expectations and maintain a state of flow.

1.2 Designing the New Determinate Progress Flow: A Narrative of Action

The core principle of the redesign is to replace the indeterminate spinner with a determinate `ProgressView`, supplemented by descriptive text that narrates the ongoing process.⁶ This approach aligns directly with the HIG recommendation to "use a determinate progress indicator" whenever the duration of a task can be estimated.¹ The entire video import and save flow will be framed as a sequence of clear, distinct stages, transforming passive waiting time into an informative and reassuring experience.⁵

To achieve this, the underlying system operations will be mapped to discrete UI states. This practice of showing distinct steps, rather than a simple, monolithic percentage, is a recognized best practice for complex, multi-stage tasks, as it provides a much clearer mental model for the user.³ The following table serves as the formal specification for this mapping, ensuring a direct and verifiable link between backend processes and the user-facing presentation. This mapping provides clear, testable acceptance criteria; for instance, an automated UI test can verify that when the underlying

`AVAssetExportSession` reports 50% completion, the on-screen `ProgressView` displays a value within the 10-60% range and the label reads "Applying edits...". This formal linkage eliminates ambiguity for the implementing engineer and guarantees the final product meets the design intent with precision.

System Stage	Morphism Chain	Progress Range	User-Facing Message	Technical Notes & Key APIs
Initialization	<code>selectVideo</code>	0-10%	"Preparing video import..."	Initial <code>PHAsset</code> loading. This

				stage is typically brief and may begin as indeterminate before switching to a determinate state, a practice endorsed by the HIG. ¹
Processing	applyEdits (applyTrim ◦ applyRotation)	10-60%	"Applying edits..."	This is the most computationally intensive stage. Progress is directly and smoothly tied to the progress property of the AVAssetExport Session.
Persistence	saveToPhotos	60-90%	"Saving to BreakDex album..."	This stage is I/O bound. It involves finding or creating the PHAssetCollection and executing the PHAssetChangeRequest. Progress here is less granular and may appear to

				jump.
Finalization	persistToCore Data	90-100%	"Finalizing..."	A quick final operation to save the PHAsset.localIdentifier and other metadata to the CoreData database.

1.3 Engineering Perceived Performance and Trust

A crucial aspect of the user experience is managing the perception of time. The HIG explicitly warns against deceptive progress reporting, such as a progress bar that rapidly fills to 99% and then stalls for an extended period.¹ This pattern erodes user trust and creates frustration.³ The staged approach defined above is designed to mitigate this risk. The "Processing" stage (10-60%), driven by the continuous

progress property of AVAssetExportSession, will be the longest but also the smoothest. The subsequent "Persistence" and "Finalization" stages are inherently faster. This structure creates a natural perception of acceleration towards completion, which has been shown to make the overall wait time feel shorter and more pleasant.³

To further enhance the user's sense of control, the HIG recommends allowing users to halt processing where feasible.¹ A "Cancel" button will be incorporated into the loading view. This requires careful state management. If the user initiates a cancellation during the

AVAssetExportSession (the "Processing" stage), a call to cancelExport() will terminate the operation cleanly. However, if the cancellation is requested during the PHPhotoLibrary.performChanges block (the "Persistence" stage), the operation is atomic and cannot be safely interrupted. During this critical, non-interruptible phase, the "Cancel" button will be temporarily disabled. This state-dependent control provides users with agency without risking data corruption or an inconsistent application state.

Ultimately, the loading bar is more than a transient UI element; it is a recurring touchpoint that builds and reinforces user trust. For the "Athlete" persona, who will perform this action repeatedly, a consistent, reliable, and transparent loading experience builds a predictable

mental model of the app's performance. By accurately reflecting the system's internal stages, the user learns to anticipate the duration of each step, reducing anxiety on subsequent uses. This well-architected loading experience reinforces the app's core value proposition as a precise, reliable, and professional-grade tool, making its design integral to long-term user trust and retention.

Part II: A Categorical Model of the Video Persistence Pipeline

To ensure the correctness and robustness of the underlying system, the architecture will be formalized using the language of category theory. This abstract model allows for reasoning about the system's behavior with mathematical precision, moving beyond informal descriptions to a verifiable blueprint. This approach helps to "find symmetries, or inconsistencies, or other kinds of patterns that might otherwise go unnoticed".⁹

2.1 The Ideal Category (Cideal): A Model of Correctness

The ideal category, denoted as *Cideal*, represents the system as it *should* be. In this model, every operation is pure, total (defined for all valid inputs), and composes predictably.

- **Objects (Data States):** These are the well-defined data structures that represent the state of the system at various points in the workflow.
 - PHAsset: The initial, unedited asset selected from the user's library.
 - EditParams: A tuple representing the user's desired edits, specifically (CMTimeRange, CGAffineTransform) for the trim and rotation.
 - TempVideoURL: The local file URL of the processed video after it has been exported by AVAssetExportSession.
 - SavedPHAsset: The new PHAsset instance created in the "BreakDex" album, which contains the processed video data. The existence and properties of this object serve as the ultimate source of truth.
 - MoveEntity: The NSManagedObject instance in the CoreData store, containing the move's name and a stable, reliable reference to the SavedPHAsset.
- **Morphisms (State Transitions):** These are the functions that transition the system between states.
 - processVideo: (PHAsset, EditParams) → TempVideoURL: A function wrapping AVAssetExportSession that applies the trim and rotation. This morphism must be an

isomorphism with respect to the video's visual representation; the output file must be a perfect visual replica of the user's edits (WYSIWYG).

- `findOrCreateAlbum`: `String` $\rightarrow$ `PHAssetCollection`: An idempotent function that always returns the "BreakDex" album, creating it only if it does not already exist.
- `saveToAlbum`: `(TempVideoURL, PHAssetCollection)` $\rightarrow$ `SavedPHAsset`: A function wrapping `PHPhotoLibrary.performChanges` to create a new asset within the target album.
- `createMoveRecord`: `(SavedPHAsset, String)` $\rightarrow$ `MoveEntity`: A function that creates the `CoreData` record, storing a stable identifier (e.g., `localIdentifier` and/or `cloudIdentifier`) from the `SavedPHAsset`.

The correctness of this ideal system can be verified using commutative diagrams.¹⁰ The entire end-to-end flow is a composition of these morphisms. Let the initial input be

`Input = (PHAsset, EditParams, String)`. The main composed morphism is `flow = createMoveRecord ◦ (saveToAlbum ◦ (processVideo, findOrCreateAlbum))`. A verification function, `verify: MoveEntity  $\rightarrow$  Bool`, can check if the video referenced by the entity visually matches the original `EditParams`. For the system to be correct, the diagram must commute, meaning `verify(flow(Input))` must always evaluate to true.

2.2 The Observed Category (Cobserved): Modeling the Flawed Reality

The observed category, `Cobserved`, models the current system with its potential for errors, race conditions, and data loss. In this category, objects may be ill-defined (e.g., a `MoveEntity` with a dangling reference to a non-existent asset), and morphisms are partial (fail for some inputs) or incorrect.

- **Isomorphism Failure: The Rotation Bug:** The current implementation contains a buggy morphism, `processVideo_buggy: (PHAsset, EditParams)  $\rightarrow$  TempVideoURL`. This morphism is not an isomorphism because it fails to preserve the structure of the visual data. While it correctly applies the trim (`CMTIME_RANGE`), it neglects to apply the `CGAffineTransform` for rotation. This results in a loss of user intent and breaks the WYSIWYG contract. This is a classic example of a morphism failing to preserve the essential structure of its objects.¹²
- **Non-Commutative Diagram: The Album Creation Race Condition:** A critical flaw exists in the logic for saving to the "BreakDex" album. A naive implementation follows the sequence: (1) fetch album, (2) check if it exists, (3) if not, create album, and (4) save the video. The fetch, create, and save operations are all asynchronous calls to `PHPhotoLibrary`. If a user adds two moves in quick succession, two concurrent threads can execute this logic. Both threads might simultaneously determine that the album does

not exist and both proceed to issue a creation request. This results in two "BreakDex" albums, a direct violation of the idempotency requirement.

In categorical terms, this race condition manifests as a non-commutative diagram. The composition of morphisms becomes order-dependent and is not associative under concurrency, which violates a fundamental axiom of a category.¹² The path $(\text{createAlbum}_1 \circ \text{findAlbum}_1)$ followed by $(\text{saveVideo}_2 \circ \text{findAlbum}_2)$ does not lead to the same final state as a correct, sequential execution. This non-commutativity provides a formal, mathematical description of the race condition bug.

2.3 The Functor of Failure ($F:\text{Cideal} \rightarrow \text{Cobserved}$): Pinpointing the Faults

A functor is a map between categories that preserves their fundamental structure (identity morphisms and composition).¹² The buggy implementation can be modeled as a functor,

F , that maps the components of the ideal category to the flawed components of the observed one.

- $F(\text{processVideo})$ maps the ideal video processing function to $\text{processVideo_buggy}$, which fails the isomorphism property.
- $F(\text{findOrCreateAlbum})$ maps the ideal, atomic, idempotent function to a non-atomic, racy sequence of operations.

The bugs in the system can be precisely identified as the points where the functor F **fails to preserve the structure** of Cideal . For example, the composition of findAlbum and createAlbum in Cideal is a single, atomic operation. The functor F maps this to a sequence of separate, non-atomic operations in Cobserved . The composition of these mapped operations is broken under concurrency, meaning that $F(g \circ f) \neq F(g) \circ F(f)$. Analyzing these violations of the functor laws provides a precise, formal "bug report" that pinpoints exactly where and how the implementation deviates from the correct behavior.

Part III: Diagnosis and Rectification via Natural Transformations

This section bridges the gap between the abstract diagnosis derived from the categorical model and the concrete engineering solutions required to fix the system. Categorical

constructs will be used to define the "patches" needed to transform the observed, flawed system into one that behaves identically to the ideal model.

3.1 Root Cause Analysis: From Broken Functors to Flawed Code

The analysis of the "Functor of Failure" reveals several classes of defects in the current implementation.

- **Missing Morphisms (Unhandled Errors):** The current workflow lacks clearly defined morphisms for handling failure states. For example, there is no specified transition from an InProgress state to a Failed state if the AVAssetExportSession encounters an error or if PHPhotoLibrary access is revoked mid-operation. This leaves the system in an undefined state. In Swift, these missing pathways can be explicitly modeled and handled using monads like Result or Optional, which have a strong categorical foundation for managing computations that can fail.¹⁴
- **Inefficient Functors (Suboptimal Flows):** The naive "check-then-create" logic for album management represents an inefficient functor. It is not only prone to race conditions but also requires multiple, separate asynchronous round-trips to the Photos library, which is suboptimal from a performance perspective.
- **Object Gaps (Incomplete State):** The MoveEntity object in the observed category, Cobserved, is incomplete. It likely stores only the PHAsset.localIdentifier, which is an insufficient representation of the asset's identity in an environment where iCloud Photos is active. This object is missing a crucial property: the cloudIdentifier.

3.2 Hypothesizing Solutions with Categorical Constructs

The solutions to these problems can be conceptualized using higher-level categorical constructs, which then guide the implementation of specific, provably correct code patterns.

- **Natural Transformations as Provable Fixes:** A natural transformation provides a systematic way to map one functor to another while respecting the underlying structure of the categories involved.¹³ The goal is to define a natural transformation, $\eta: F \Rightarrow G$, where F is the buggy "Functor of Failure" and G is a new, correct functor (ideally, the identity functor, which maps Cideal to itself). For example, the incomplete CoreData model can be fixed with such a transformation. The current functor F maps SavedPHAsset $\rightarrow$ MoveEntity by only copying the localIdentifier, resulting in an object we can call MoveEntity_localOnly. A correct functor G would map it to MoveEntity_robust, which stores both localIdentifier and cloudIdentifier.

The fix is a natural transformation η that defines a morphism $\eta_{\text{SavedPHAsset}}: \text{MoveEntity_localOnly} \rightarrow \text{MoveEntity_robust}$. This morphism represents a function that takes the incomplete entity, fetches the corresponding PHAsset, retrieves its `cloudIdentifier`, and populates the new field. By applying this transformation, the flawed implementation (F) is systematically and consistently upgraded to a correct one (G).

- **Adjoint Functors for Idempotent Album Creation:** The "find or create" pattern is a universal construction problem. In category theory, such problems are often solved elegantly by **adjoint functors**. While a deep dive into the mathematics is unnecessary here, the practical implication is that an adjoint provides the "most efficient" or "universal" way to construct a mapping. This abstract concept translates directly to a concrete best practice: using the `PHPhotoLibrary.performChanges` block to its full potential. Within a single, atomic change block, one can request the creation of a new asset and simultaneously request that this new asset be added to a target album (which can be referenced by a placeholder if it's also being created in the same block). The Photos framework guarantees the atomicity of this entire transaction, ensuring that if any part fails, the entire operation is rolled back. This is the concrete implementation of the abstract universal property, guaranteeing both correctness and efficiency.
- **Categorical Limits for Robust Testing:** The formal model also informs a more rigorous testing strategy. A **product** in category theory is a way of combining objects. The categorical product of all possible user inputs (e.g., different PHAsset types, variations of `EditParams`, different permission states) defines the total state space that must be covered by testing. Dually, a **colimit** can represent the merging of different execution paths (e.g., success, cancellation, export failure, save failure) into a final, observable state. By structuring integration tests around these concepts of limits and colimits, it is possible to ensure that all structurally significant paths through the system are tested, moving beyond a few hand-picked "golden path" examples.

Part IV: SPEC-KIT: A Formal Specification for Implementation

This section provides a sequential, verifiable checklist for a senior agentic principal engineer. It translates the preceding formal analysis into actionable engineering tasks, adhering to Spec-Driven Development principles to emphasize verifiable intent and reduce ambiguity.

4.1 Vision & Principles ("What" & "Why")

- **Goal:** To implement a WYSIWYG, fault-tolerant video saving flow, fronted by a best-in-class, HIG-compliant progress indicator that enhances user trust and provides clear system feedback.
- **User Persona & Journey:** The primary user, "The Athlete," selects a video, applies precise trim and rotation edits, names the move, and saves it. The user expects the saved video to be a perfect replica of their edits and for the saving process to be transparent, predictable, and reliable.
- **Non-Goals:** This feature will not include advanced video filtering (e.g., color correction), the merging of multiple clips, or cloud-based transcoding. The scope is strictly limited to the local device processing and persistence pipeline.

4.2 Verifiable Requirements & Acceptance Criteria

- **AC1 (UI):** The loading screen MUST display a determinate `ProgressView` and a `Text` label. The progress value and text message MUST update according to the stages defined in the System Stage to UI State Mapping table.
- **AC2 (Permissions):** The application MUST request `PHPhotoLibrary` read/write access before presenting the photo picker. If access is denied or restricted, an alert MUST be displayed that guides the user to the system Settings app.¹⁵
- **AC3 (Video Processing):** The final saved video asset in the Photos library MUST have the exact `CMTimeRange` (trim) and `CGAffineTransform` (rotation) applied as specified by the user in the `NameMoveView`. This MUST be visually verifiable (WYSIWYG).
- **AC4 (Photos Album Idempotency):** The system MUST save the processed video to a `PHAssetCollection` titled "BreakDex". If this album does not exist, it MUST be created. If it already exists, the existing album MUST be used. The system MUST NEVER create a duplicate "BreakDex" album.
- **AC5 (CoreData Integrity):** Upon a successful save to the Photos library, a `MoveEntity` MUST be created in the `CoreData` store. This entity MUST store a stable identifier for the new `PHAsset`. The video relationship in the `MoveDetailView` MUST resolve correctly to display the saved video.
- **AC6 (Fault Tolerance):** If the user manually deletes the video from the Photos app after it has been saved, the `MoveDetailView` MUST NOT crash. It should gracefully handle the missing asset, for example, by displaying a placeholder and a message indicating the video is no longer available.
- **AC7 (Cancellation):** The user MUST be able to cancel the operation during the "Processing" stage via a "Cancel" button. A successful cancellation MUST result in no new video being saved to the Photos library and no new `MoveEntity` being created in `CoreData`.

4.3 Sequential Implementation Checklist

Step 1: Permissions & Info.plist Setup

1. **Configure Info.plist:** Ensure the `NSPhotoLibraryUsageDescription` key is present in Info.plist. The associated string value should provide a clear, user-friendly explanation of why the app requires access to the photo library.¹⁶
2. **Implement PermissionManager:** Create a dedicated utility or service to encapsulate all interactions with `PHPhotoLibrary` authorization. This service should provide a single, asynchronous method to check and request permission. It must correctly handle all possible `PHAuthorizationStatus` cases: `.authorized`, `.limited`, `.denied`, and `.notDetermined`.¹⁵
3. **Gate the Feature:** The entry point to the entire "add move" flow must be gated by a successful permission check from the `PermissionManager`.

Step 2: UI Layer - The Progress View

1. **Create ProgressView:** Develop a new SwiftUI view, `ProgressView.swift`, which will serve as the loading screen. This view should be driven by a dedicated `ProgressViewModel`. The `ViewModel` will expose `@Published` properties for the progress value (`Double`) and the status message (`String`).
2. **Utilize Standard Components:** The view should use a standard `SwiftUI.ProgressView(value: progress)` and a `Text(message)` view to display the information from the `ViewModel`.
3. Integrate into Flow:
`GrepTool(pattern="NameMoveViewModel", path="Views/NameMoveView.swift")`
Present the `ProgressView` as a full-screen modal immediately upon the user tapping the "Save" button in the `NameMoveView`. The `NameMoveViewModel` will be responsible for instantiating and managing the `ProgressViewModel`, which will orchestrate the entire save process.

Step 3: AVFoundation - The Processing Morphism

1. **Create VideoProcessor Service:** Implement a VideoProcessor.swift service. This service will contain the primary method: processVideo(asset: PHAsset, trimRange: CMTimeRange, rotationTransform: CGAffineTransform) async throws -> URL. This function will return the URL of the temporary, processed video file.
2. **Implement Correct Transformation:** This method must use AVAssetExportSession. The key to fixing the rotation bug and achieving WYSIWYG is to construct an AVMutableVideoComposition and an AVMutableVideoCompositionLayerInstruction. The user-specified rotationTransform must be correctly applied to the layer instruction's transform property.²¹
3. **Publish Progress:** The method must monitor the progress property of the AVAssetExportSession instance. It should publish these progress updates (e.g., via an AsyncStream or a closure) so the ProgressViewModel can consume them and update the UI in real-time.

Swift

// Conceptual Snippet for VideoProcessor.swift

```
func processVideo(...) async throws -> URL {  
    //... setup AVAsset and tracks...  
    let videoComposition = AVMutableVideoComposition()  
    videoComposition.renderSize = //... correctly calculate rotated size...  
    videoComposition.frameDuration = CMTime(value: 1, timescale: 30)  
  
    let instruction = AVMutableVideoCompositionLayerInstruction(assetTrack: videoTrack)  
    // This is the critical step for WYSIWYG rotation  
    instruction.setTransform(rotationTransform, at:.zero)  
  
    let mainInstruction = AVMutableVideoCompositionInstruction()  
    mainInstruction.timeRange = CMTimeRange(start:.zero, duration: asset.duration)  
    mainInstruction.layerInstructions = [instruction]  
    videoComposition.instructions = [mainInstruction]  
  
    guard let exportSession = AVAssetExportSession(...) else { /* throw error */ }  
    exportSession.videoComposition = videoComposition  
    exportSession.timeRange = trimRange  
    exportSession.outputURL = /* temporary URL */  
    exportSession.outputFileType = .mp4  
  
    //... await exportSession.export() while monitoring progress...  
    return outputURL  
}
```

Step 4: PhotosKit - The Persistence Morphism

1. **Create PhotosService:** Implement a PhotosService.swift with a primary method: saveVideo(from temporaryURL: URL) async throws -> String. This method will return the localIdentifier of the newly created PHAsset.
2. **Ensure Atomicity:** This method must implement the robust "find or create album" logic inside a single PHPhotoLibrary.shared().performChanges block. This ensures the entire operation is atomic and prevents the race condition identified in Part II.²⁴

Swift

// Conceptual Snippet for PhotosService.swift

```
func saveVideo(...) async throws -> String {
    var placeholder: PHObjectPlaceholder?

    try await PHPhotoLibrary.shared().performChanges {
        let album = self.findOrCreateAlbum(named: "BreakDex") // Must be synchronous within
        this block

        guard let creationRequest =
        PHAssetChangeRequest.creationRequestForAssetFromVideo(atFileURL: temporaryURL)
        else { return }

        if let album = album, let albumChangeRequest = PHAssetCollectionChangeRequest(for:
        album) {
            let assetPlaceholder = creationRequest.placeholderForCreatedAsset!
            albumChangeRequest.addAssets([assetPlaceholder] as NSFastEnumeration)
        }
        placeholder = creationRequest.placeholderForCreatedAsset
    }

    guard let localIdentifier = placeholder?.localIdentifier else { /* throw error */ }
    return localIdentifier
}
```

Step 5: CoreData - The Finalization Morphism

1. Locate CoreData Stack:

GrepTool(pattern="PersistenceController", path="Data/Persistence.swift")

GrepTool(pattern="Move", path="Data/Model.xcdatamodeld")

2. **Update Data Model:** Ensure the Move entity in the CoreData model (.xcdatamodeld) has an attribute videoAssetIdentifier of type String.
3. **Save Entity:** The NameMoveViewModel will orchestrate the flow. After the PhotosService successfully returns a localIdentifier, the ViewModel will call the PersistenceController to create and save a new Move object on the main context. This action must only occur after the video is successfully persisted in the Photos library.
4. **Update Detail View:** The MoveDetailView will use the stored videoAssetIdentifier to fetch the corresponding PHAsset from the Photos library for display.

4.4 Identifying Blind Spots & Future Considerations

- **iCloud Sync & Identifier Stability:**
 - **Problem:** The PHAsset.localIdentifier is not stable across a user's devices. An identifier generated on an iPhone will be invalid on that user's iPad after an iCloud Photos sync, causing the video to appear missing.
 - **Solution:** The data model must be made more robust, as proposed in Part III. The Move entity in CoreData requires a second, optional attribute: videoAssetCloudIdentifier of type String. During the save process, after retrieving the localIdentifier, a subsequent fetch must be made to retrieve the asset's cloudIdentifier (if available) and store both.
 - **Reconciliation Logic:** The data fetching logic in MoveDetailView must be updated to be fault-tolerant. It should first attempt to fetch the PHAsset using the localIdentifier. If this fetch fails (returns zero assets), it must perform a second fetch using the cloudIdentifier. This two-step fetch process creates a robust system that works seamlessly with iCloud Photos and correctly implements the conceptual "natural transformation" fix.
- **Background Execution:** An AVAssetExportSession for a large video file can take a significant amount of time. If the user backgrounds the application during this process, the export may be suspended or terminated by the OS. For a future iteration, the use of a BGProcessingTaskRequest should be investigated to allow the export to complete reliably in the background.
- **Large File Handling & Memory Management:** Processing large 4K or high-framerate videos can consume considerable memory and temporary disk space. The implementation must be diligent about cleaning up the temporary file located at the TempVideoURL. This cleanup must occur reliably after the video has been successfully saved to the Photos library, as well as in all failure and cancellation scenarios, to prevent storage leaks.
- **"Limited Library" Access:** iOS 14 introduced a "Limited Library" access mode, which

allows users to grant an app access to only a subset of their photos.²⁰ The permission checks must gracefully handle this .limited status. If the user has granted limited access and the video they wish to import is not part of the selected set, the application should not fail silently. Instead, it should use the `PHPhotoLibrary.presentLimitedLibraryPicker(from:)` API to allow the user to modify their selection or grant full access.²⁰

Works cited

1. Progress indicators | Apple Developer Documentation, accessed September 27, 2025, <https://developer.apple.com/design/human-interface-guidelines/progress-indicators>
2. Progress Indicators - UI Controls - iOS Human Interface Guidelines - CodersHigh, accessed September 27, 2025, <https://codershigh.github.io/guidelines/ios/human-interface-guidelines/ui-controls/progress-indicators/index.html>
3. Best Practices For Animated Progress Indicators - Smashing Magazine, accessed September 27, 2025, <https://www.smashingmagazine.com/2016/12/best-practices-for-animated-progress-indicators/>
4. Progress Bar Indicator UX/UI Design & Feedback Notifications - Usersnap, accessed September 27, 2025, <https://usersnap.com/blog/progress-indicators/>
5. Loading - Interaction - iOS Human Interface Guidelines - CodersHigh, accessed September 27, 2025, <https://codershigh.github.io/guidelines/ios/human-interface-guidelines/interaction/loading/index.html>
6. Displaying progress views and gauges | Apple Developer Documentation, accessed September 27, 2025, <https://developer.apple.com/documentation/clockkit/displaying-progress-views-and-gauges>
7. Loading Bar Design: Do's and Don'ts You Should Know, accessed September 27, 2025, <https://wpamelia.com/loading-bar/>
8. Progress Bar Design Best Practices | by uxplanet.org, accessed September 27, 2025, <https://uxplanet.org/progress-bar-design-best-practices-526f4d0a3c30>
9. Apply category theory to modeling and software development, accessed September 27, 2025, <https://www.johndcook.com/blog/applied-category-theory/>
10. Getting Started with Category Theory - Ryan Brewer, accessed September 27, 2025, <https://ryanbrewer.dev/posts/getting-started-category-theory.html>
11. Lesson 49: Introduction to Category Theory - What is Functional Programming?, accessed September 27, 2025, <https://www.kindafunctional.com/lesson-49-introduction-to-category-theory.html>
12. An introduction to Category Theory for Software Engineers - Computer Science, accessed September 27, 2025,

- <https://www.cs.toronto.edu/~sme/presentations/cat101.pdf>
13. Introduction to category theory for programmers - Ada Beat, accessed September 27, 2025,
<https://adabeat.com/fp/introduction-to-category-theory-for-programmers/>
 14. An introduction to monads in Swift, accessed September 27, 2025,
<https://swiftstudent.com/blog/2025-5-15-an-introduction-to-monads-in-swift/>
 15. PHPhotoLibrary | Apple Developer Documentation, accessed September 27, 2025, <https://developer.apple.com/documentation/Photos/PHPhotoLibrary>
 16. Querying the iOS Photo Library - Kyle Howells, accessed September 27, 2025,
<http://ikyle.me/blog/2025/querying-the-ios-photo-library>
 17. Swift how to show alert to request permission for photo library? - Stack Overflow, accessed September 27, 2025,
<https://stackoverflow.com/questions/50122080/swift-how-to-show-alert-to-request-permission-for-photo-library>
 18. Determine if the access to photo library is set or not - PHPhotoLibrary - Stack Overflow, accessed September 27, 2025,
<https://stackoverflow.com/questions/26595343/determine-if-the-access-to-photo-library-is-set-or-not-phphotolibrary>
 19. Requesting authorization to capture and save media | Apple Developer Documentation, accessed September 27, 2025,
<https://developer.apple.com/documentation/avfoundation/requesting-authorization-to-capture-and-save-media>
 20. Handle the Limited Photos Library in your app | Documentation - WWDC Notes, accessed September 27, 2025,
<https://wwdcnotes.com/documentation/wwdcnotes/wwdc20-10641-handle-the-limited-photos-library-in-your-app/>
 21. how to export video asset via AVAssetExportSession in portrait mode - Stack Overflow, accessed September 27, 2025,
<https://stackoverflow.com/questions/10034337/how-to-export-video-asset-via-avassetexportsession-in-portrait-mode>
 22. Rotate an AVAsset with AVAssetExportSession - Stack Overflow, accessed September 27, 2025,
<https://stackoverflow.com/questions/12410130/rotate-an-avasset-with-avassetexportsession>
 23. How to Trim and Crop Video in Swift | IMG.LY Blog, accessed September 27, 2025,
<https://img.ly/blog/trim-and-crop-video-in-swift/>
 24. How to download & save the video in device storage in swift | by ..., accessed September 27, 2025,
<https://mdcode2021.medium.com/how-to-download-save-the-video-in-device-storage-in-swift-a1129aecd736>